

GapCheck

AI job-fit platform with explainable resume scoring and a Terraform-defined AWS deployment

Role: Full-stack and infrastructure engineer

Claude API

FastAPI

PostgreSQL

SQLAlchemy

Alembic

React

TypeScript

Terraform

AWS

GitHub Actions

Built a full-stack platform that uses multi-step Claude API prompt chains to turn resumes and job postings into structured data, score fit across five weighted dimensions, and explain the gaps. I also wrote the full AWS deployment in Terraform behind a credential-free CI pipeline.

5

Scoring Dimensions

Skills, experience, education, projects, domain

5

HIGH/CRITICAL Findings Fixed

Static-analysis issues in the Terraform config

0

Cloud Credentials in CI

Mocked-provider tests, tfint, and Trivy on every PR

Problem

Students waste time filtering job postings and rarely know which skills are actually missing for the roles they want. Generic AI feedback is vague and sometimes malformed.

Solution

GapCheck parses both sides into structured data, validates every AI response, and produces weighted, explainable scores and ranked gaps. Missing skills turn into concrete portfolio-project recommendations.

Architecture

- React + TypeScript frontend: job board, resume profile, match verdict, tracker
- FastAPI REST endpoints with SQLAlchemy models and Alembic migrations on PostgreSQL
- Claude API prompt chains with schema validation on every response
- Pluggable job-source adapters and an Airflow ingestion pipeline for repeatable updates
- Terraform: ECS Fargate behind an ALB, RDS PostgreSQL, VPC, IAM, CloudWatch
- GitHub Actions: mocked-provider tests, tfint, and Trivy security scanning

Highlights

- Multi-step Claude API prompt chains extract structured JSON from resumes and job postings
- Backend validation rejects malformed AI responses before they reach users
- Weighted, explainable compatibility scores with frequency-weighted required skills
- Full AWS deployment in Terraform: ECS Fargate, ALB, RDS PostgreSQL, VPC, IAM, CloudWatch
- Credential-free CI runs a mocked-provider test suite, tfint, and Trivy on every pull request
- Separated liveness from readiness probes after finding the readiness check shared the app's connection pool

Challenges and Tradeoffs

- Challenge: LLM output was sometimes malformed. Change: Schema-validated every response and rejected bad ones before they reached users.
- Challenge: Static analysis flagged 5 HIGH/CRITICAL issues in the infrastructure. Change: Hardened the Terraform and locked the fixes in as regression tests.
- Challenge: The readiness probe shared a connection pool with app traffic, so load could fail health checks. Change: Split liveness from readiness.
- Tradeoff: Real cloud credentials in CI vs safety. Decision: A mocked-provider suite, so pull requests never need AWS keys.

Outcomes

- One workflow from resume upload to ranked, explainable job matches
- Infrastructure is reviewable code, with production decisions captured as tests
- Missing skills become concrete project recommendations with difficulty, timeline, and stack

Live demo: <https://gapcheck-1.onrender.com> · Repository: <https://github.com/Jaskeerat/gapcheck> · Full case study: <https://jaskeerat-portfolio.vercel.app/projects/gapcheck>